



Anaconda и Python

проф. кафедры Эконометрики и математической экономики ЭМИТ РАНХиГС
д.т.н. Шилин Кирилл Юрьевич

РАНХиГС каб. 404/4

email: kshilin@ranepa.ru

почта для домашних заданий

email: kirill.shilin@gmail.com

Дистрибутив Anaconda



Загрузить дистрибутив Anaconda: <https://www.anaconda.com/products/individual>



Версия Python 3.8

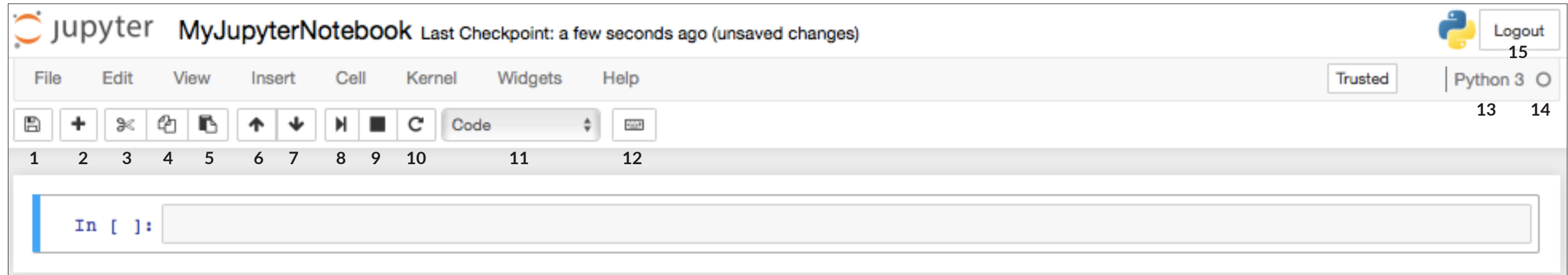


Редактор Jupyter Notebooks версия 6 и старше

Jupyter Notebooks

Шпаргалка (Cheat Sheet) для Jupyter Notebook [ссылка](#)

Command Mode:



Saving/Loading Notebooks

Create new notebook

Make a copy of the current notebook

Save current notebook and record checkpoint

Preview of the printed notebook

Close notebook & stop running any scripts

Open an existing notebook

Rename notebook

Revert notebook to a previous checkpoint

Download notebook as

- IPython notebook
- Python
- HTML
- Markdown
- reST
- LaTeX
- PDF

Insert Cells

Add new cell above the current one

Add new cell below the current one

Restart kernel

Restart kernel & run all cells

Restart kernel & run all cells

Interrupt kernel

Interrupt kernel & clear all output

Connect back to a remote notebook

Run other installed kernels

Jupyter Notebooks

Основные возможности:

1. Поля для ввода кода
2. Поблочное выполнение кода
3. Поля для ввода заголовков
4. Поля для ввода текста с формулами LaTeX

Внимание! команда для выполнения блока **<Shift>+<Enter>**

Ввод данных в режиме **Markdown**

Заголовки

Заголовок 1 уровня: **#** или **<h1>...</h1>**

Заголовок 2 уровня: **##** или **<h2>...</h2>**

Заголовок 3 уровня: **###** или **<h3>...</h3>**

Формула TeX: в выделенной строке **$\int_0^{\pi} \sin^2(x) dx$**
внутри строки **$\int_0^{\pi} \sin^2(x) dx$**

Конец абзаца: В конце абзаца поставить **
**



РАНХиГС

РОССИЙСКАЯ АКАДЕМИЯ НАРОДНОГО ХОЗЯЙСТВА
И ГОСУДАРСТВЕННОЙ СЛУЖБЫ
ПРИ ПРЕЗИДЕНТЕ РОССИЙСКОЙ ФЕДЕРАЦИИ



РАНХиГС

ЭКОНОМИЧЕСКИЙ
ФАКУЛЬТЕТ



Структурированные данные

Структурированные данные Python

Списки могут динамически расти и сокращаться до любого размера.

Объекты в списке хранятся под индивидуальными номерами.



Как и в массивах, элементы списков нумеруются с нуля, и эти номера называются «значениями индекса» или просто «индексами».

Множество можно представить в виде коллекции неупорядоченных уникальных элементов – повторов нет.



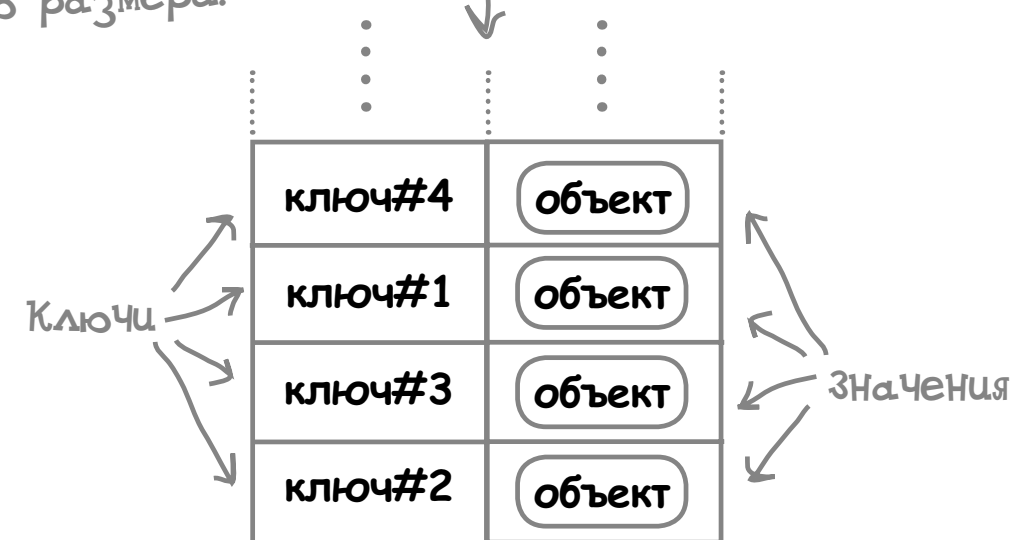
Множество

Кортежи похожи на списки, только их НЕЛЬЗЯ изменить. Кортежи – константные списки.



В кортежах также есть индексы (как и в списках).

Словари связывают ключи со значениями, и (как списки) их можно динамически сокращать или увеличивать до любого размера.



Словарь

Список в Python

```
prices = []
```

Имя переменной находится слева от оператора присваивания...

...а литеральный список – справа. В этом примере список пуст.

```
temps = [ 32.0, 212.0, 0.0, 81.6, 100.0, 45.3 ]
```

Объекты (в данном случае числа с плавающей точкой) отделены друг от друга запятыми и заключены в квадратные скобки – это список.

```
words = [ 'hello', 'world' ]
```

Список строковых объектов.

```
car_details = [ 'Toyota', 'RAV4', 2.2, 60807 ]
```

Список объектов различных типов.

```
everything = [ prices, temps, words, car_details ]
```

Списки внутри списка.

```
odds_and_ends = [ [ 1, 2, 3 ], [ 'a', 'b', 'c' ],  
                  [ 'One', 'Two', 'Three' ] ]
```

Списки НЕКОПИЯ



```
>>> first = [1, 2, 3, 4, 5]
>>> first
[1, 2, 3, 4, 5]
>>> second = first
>>> second
[1, 2, 3, 4, 5]
```

Создать Новый список
(и положить в него пять
числовых объектов).

Пять элементов в списке «first».

«Скопировать» существующий список
в Новый, который назовем «second».

Пять элементов в списке «second».

```
>>> second.append(6)
>>> second
[1, 2, 3, 4, 5, 6]
```

Кажется, что все
Нормально. Но это не так.

```
>>> first
[1, 2, 3, 4, 5, 6]
```

Упс! Новый объект
добавился также
в список «first»!



Списки КОПИЯ



```
>>> third = second.copy()  
>>> third  
[1, 2, 3, 4, 5, 6]
```



Список «third»
вырос на один
объект. →

```
>>> third.append(7)  
>>> third  
[1, 2, 3, 4, 5, 6, 7]  
>>> second  
[1, 2, 3, 4, 5, 6]
```

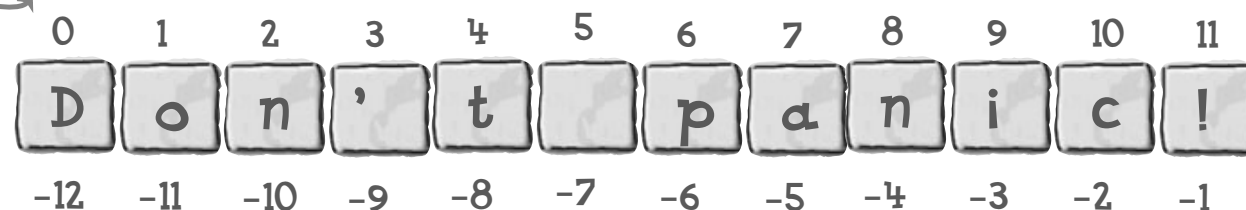


Намного лучше.
Существующий список
не изменился.

А вот это
похоже на правду.
Новый объект
добавлен только
в список «third»,
но не в остальные два
(«first» и «second»).

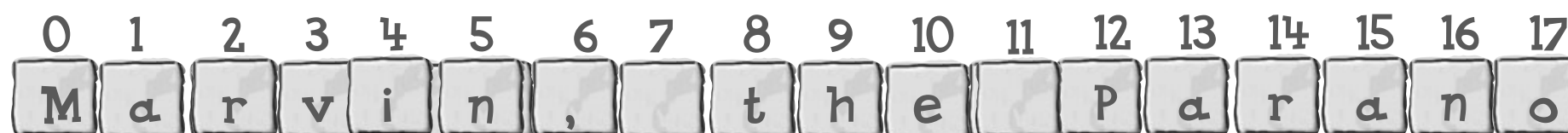
Адреса в списках

В списках Python
положительные
индексы начинаются
с 0...

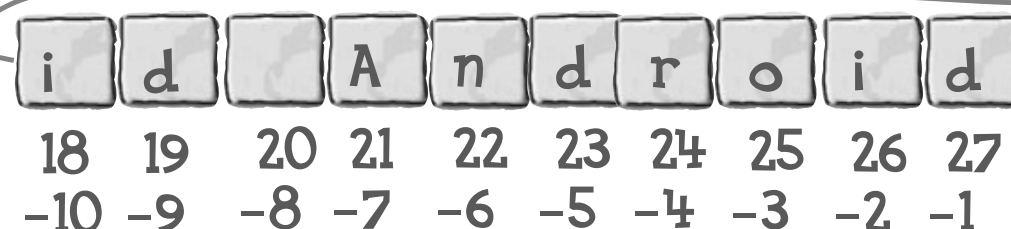


...а отрицательные
индексы начинаются
с -1.

letters



Вспомним, что
доступ к любому
объекту в списке
можно получить
по положительному
или отрицательному
индексу. Здесь
рассмотрим некоторые
отрицательные индексы.



```
for char in letters[:6]:
    print('\t', char)
```

letters[:6]



```
for char in letters[-7:]:
    print('\t'*2, char)
```

letters[-7:]

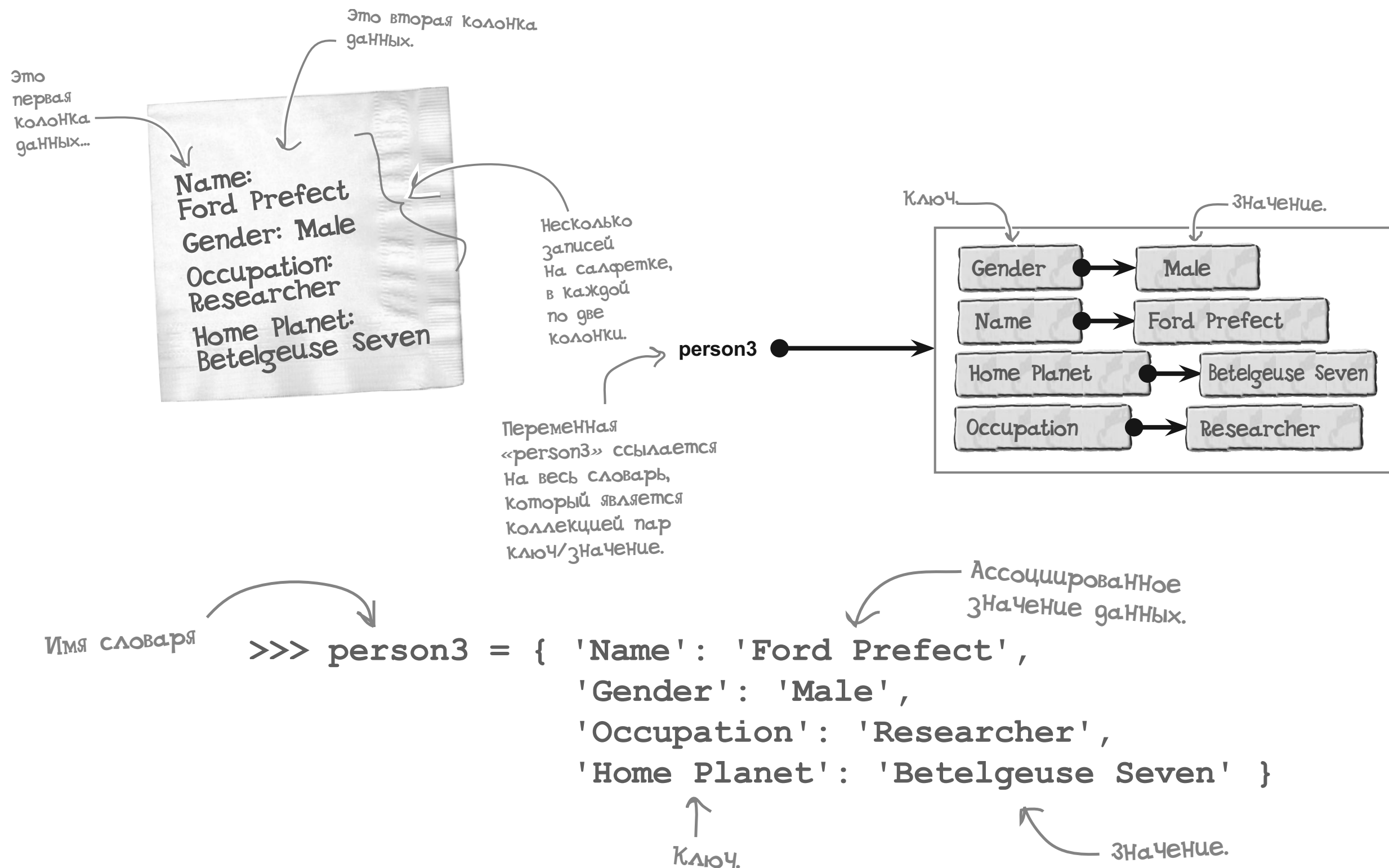


```
for char in letters[12:20]:
    print('\t'*3, char)
```

letters[12:20]



Словари в Python



Словари: отсутствие порядка

⋮	⋮
ключ №4	объект
ключ №1	объект
ключ №3	объект
ключ №2	объект

Словарь

```
>>> person3 = { 'Name': 'Ford Prefect',  
                 'Gender': 'Male',  
                 'Occupation': 'Researcher',  
                 'Home Planet': 'Betelgeuse Seven' }
```

Мы ввели данные
в словарь в этом
порядке...

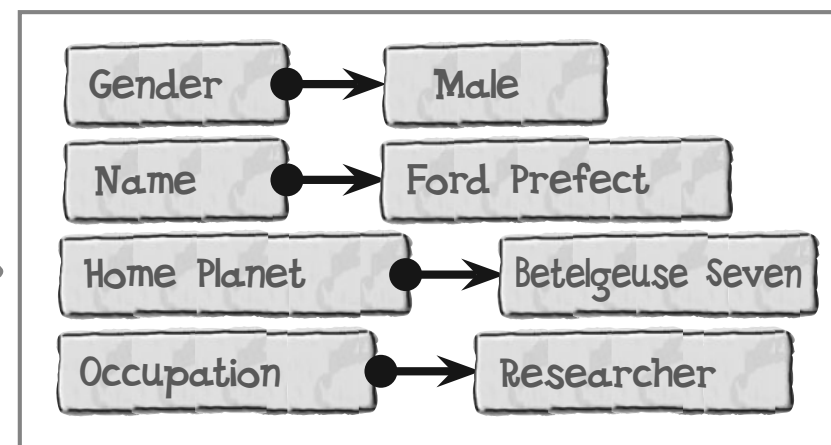
```
>>> person3
```

```
{ 'Gender': 'Male', 'Name': 'Ford Prefect', 'Home Planet':  
'Betelgeuse Seven', 'Occupation': 'Researcher' }
```

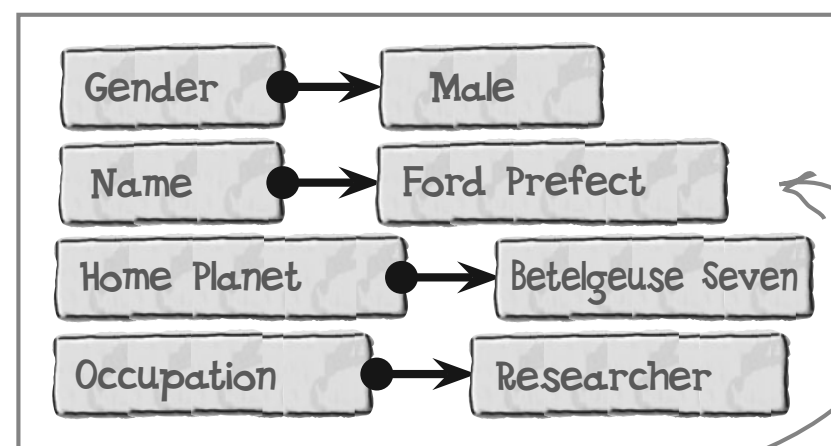
...а интерпретатор
использует другой
порядок.

Словари адресация

```
>>> person3['Home Planet']  
'Betelgeuse Seven'
```



```
>>> person3['Name']  
'Ford Prefect'
```



Укажите ключ
между квадратными
скобками.

```
>>> person3['Home Planet']  
'Betelgeuse Seven'
```

```
>>> person3['Name']  
'Ford Prefect'
```

Значение данных,
ассоциированное
с ключом.

Множества

Множество начинается и заканчивается фигурными скобками.

```
>>> vowels = { 'a', 'e', 'e', 'i', 'o', 'u', 'u' }  
>>> vowels  
{ 'e', 'u', 'a', 'i', 'o' }
```

Обратите внимание на порядок. Он отличается от порядка ввода. Также удалены повторения.

Объекты отделяются друг от друга запятыми.

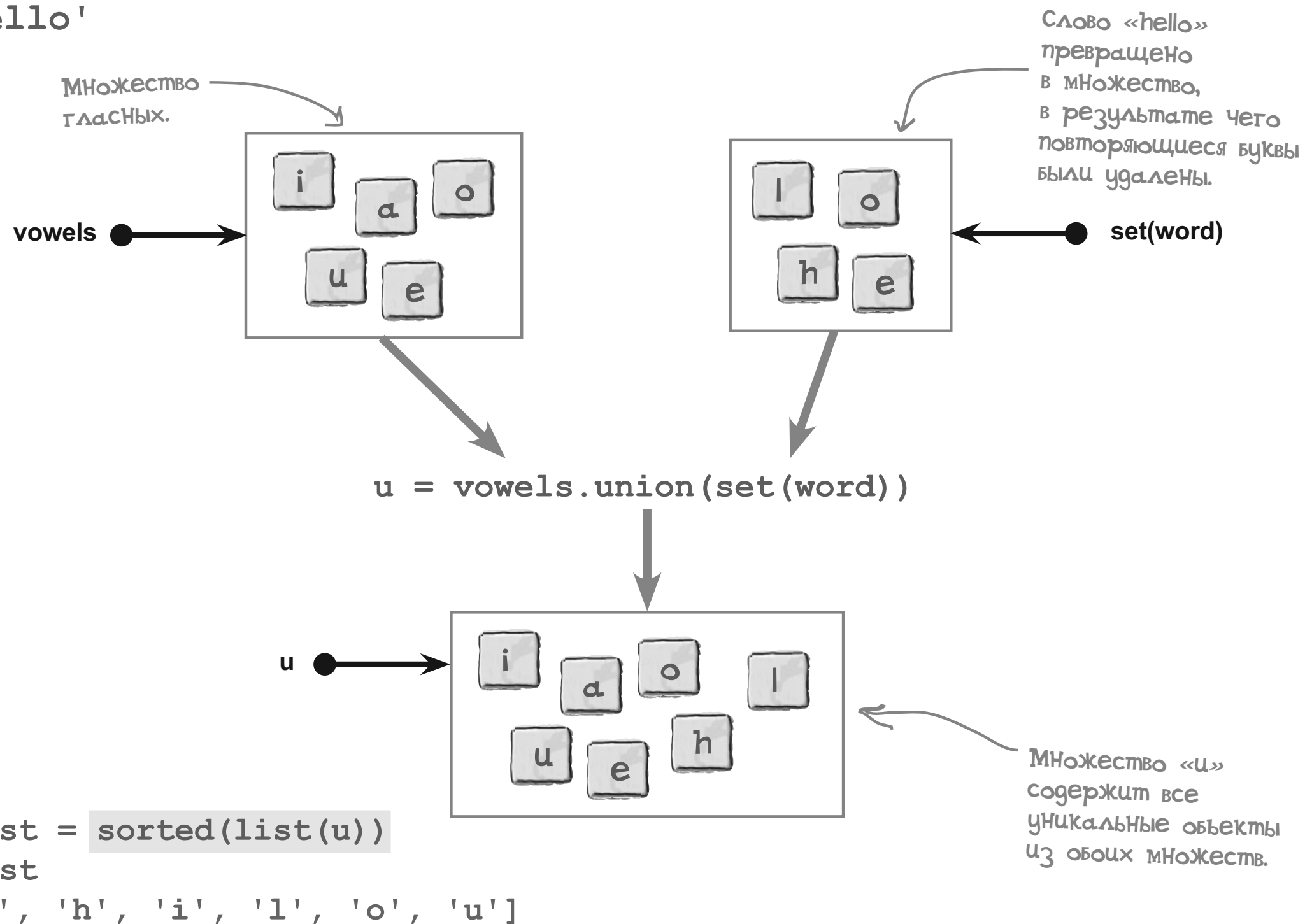
```
>>> vowels = { 'a', 'e', 'e', 'i', 'o', 'u', 'u' }  
>>> vowels  
{ 'e', 'u', 'a', 'i', 'o' }
```

Эти две строки кода делают одно и то же: они обе присваивают переменной новый объект множества.

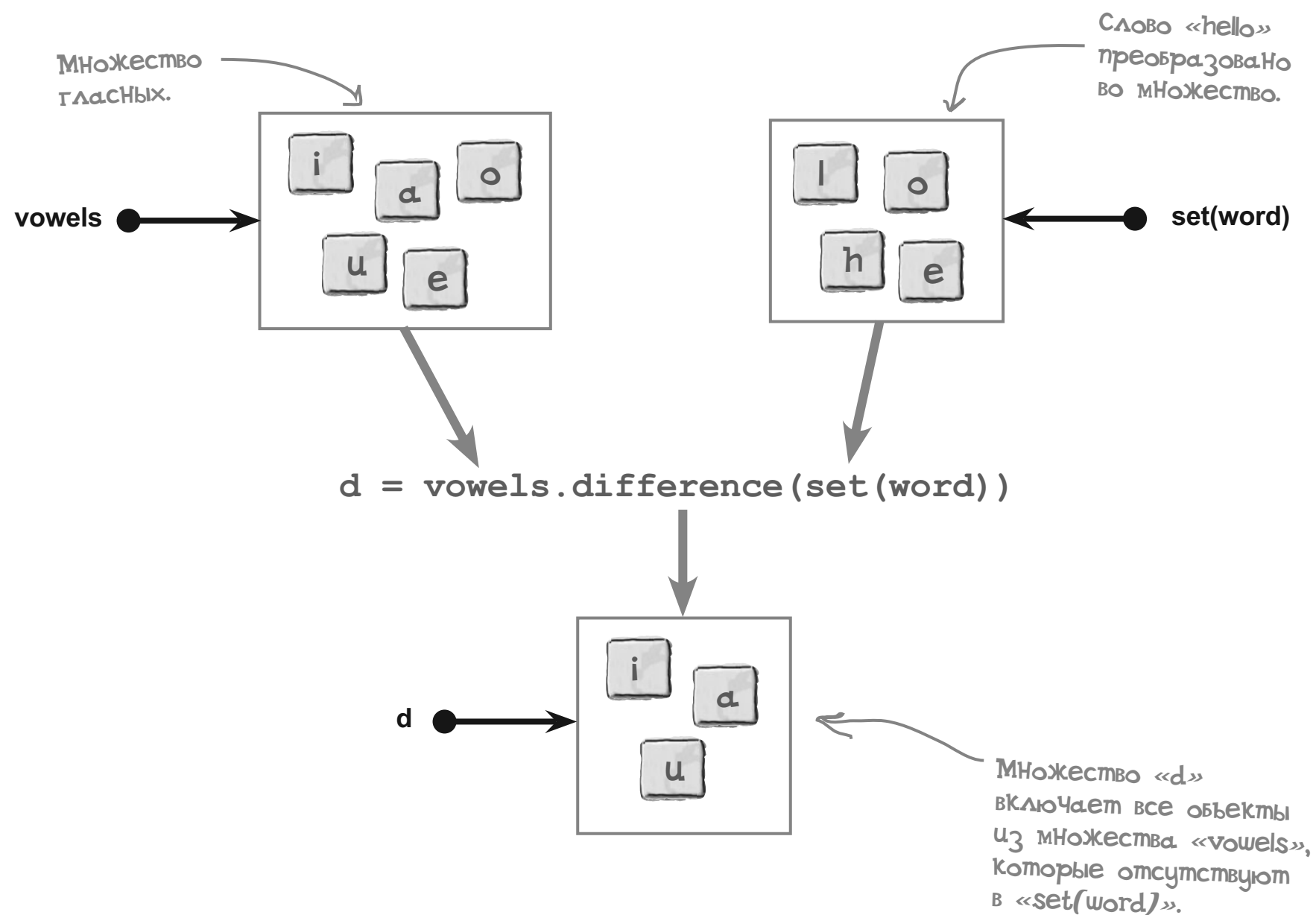
```
>>> vowels2 = set('aeiouu')  
>>> vowels2  
{ 'e', 'u', 'a', 'i', 'o' }
```

Объединение множеств

```
>>> vowels = set('aeiou')  
>>> word = 'hello'
```

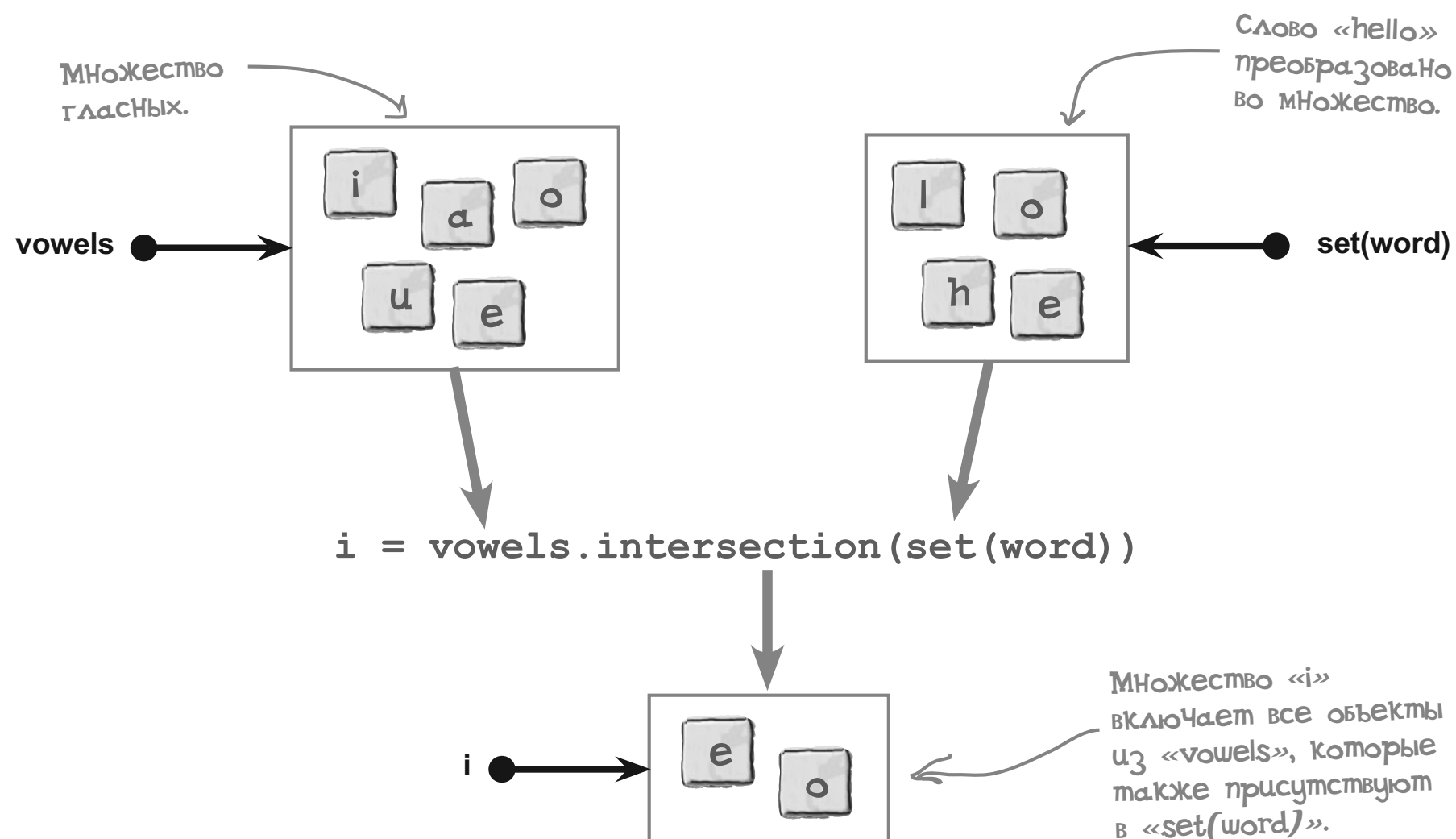


Различие в множествах



```
>>> d = vowels.difference(set(word))
>>> d
{'u', 'i', 'a'}
```

Общие объекты множеств



```
>>> i = vowels.intersection(set(word))
>>> i
{'e', 'o'}
```

Кортежи

Ничего нового.
Создан список
гласных.

```
>>> vowels = [ 'a', 'e', 'i', 'o', 'u' ]  
>>> type(vowels)  
<class 'list'>
```

Встроенная
функция «type»
сообщает тип
объекта.

```
>>> vowels2 = ( 'a', 'e', 'i', 'o', 'u' )  
>>> type(vowels2)  
<class 'tuple'>
```

Кортеж похож на список,
но не является им. Кортежи
заключены в круглые
(а не в квадратные) скобки.

```
>>> vowels  
['a', 'e', 'i', 'o', 'u']  
>>> vowels2  
( 'a', 'e', 'i', 'o', 'u' )
```

Круглые скобки показывают,
что это кортеж.

```
>>> vowels[2] = 'I'  
>>> vowels  
['a', 'e', 'I', 'o', 'u']
```

Присвоим букву «I»
верхнего регистра
третьему элементу
списка «vowels».

```
>>> vowels2[2] = 'I'  
Traceback (most recent call last):  
  File "<pyshell#16>", line 1, in <module>  
    vowels2[2] = 'I'  
TypeError: 'tuple' object does not support item assignment  
>>> vowels2  
( 'a', 'e', 'i', 'o', 'u' )
```

Интерпретатор
громко возмущается
в ответ на попытку
изменить кортеж.

Ничего не изменилось, потому
что кортежи неизменяемы.

Повторим: список и словарь

Пустой список. →

```
>>> l = list()
>>> l
[]
>>> l = [ 1, 2, 3 ]
>>> l
[1, 2, 3]
```

Используем встроенную функцию «list» для создания пустого списка, а затем присвоим данные.

Пустой словарь. →

```
>>> d = dict()
>>> d
{}
>>> d = { 'first': 1, 'second': 2, 'third': 3 }
>>> d
{'second': 2, 'third': 3, 'first': 1}
```

Используем встроенную функцию «dict» для определения пустого словаря, а затем присвоим данные.

Повторим: множество и кортеж

Пустое
множество.

```
>>> s = set()
>>> s
set()
>>> s = {1, 2, 3}
>>> s
{1, 2, 3}
```

Используем встроенную
функцию «set» для
определения пустого
множества, а затем
присвоим данные.

Пустой
кортеж

```
>>> t = tuple()
>>> t
()
>>> t = (1, 2, 3)
>>> t
(1, 2, 3)
```

Используем встроенную
функцию «tuple» для
создания пустого
кортежа, а затем
присвоим данные.

Множества
закljučаются
в фигурные скобки,
так же как словари.
Однако пара
фигурных скобок уже
используется для
представления пустого
словаря, поэтому
пустое множество
представлено как
«set()».



РАНХиГС

РОССИЙСКАЯ АКАДЕМИЯ НАРОДНОГО ХОЗЯЙСТВА
И ГОСУДАРСТВЕННОЙ СЛУЖБЫ
ПРИ ПРЕЗИДЕНТЕ РОССИЙСКОЙ ФЕДЕРАЦИИ



РАНХиГС

ЭКОНОМИЧЕСКИЙ
ФАКУЛЬТЕТ



УСЛОВИЯ И ЦИКЛЫ

Оператор условия

```
if <test1>:                # Инструкция if с условным выражением test1
    <statements1>          # Ассоциированный блок
elif <test2>:              # Необязательные части elif
    <statements2>
else:                      # Необязательный блок else
    <statements3>
```

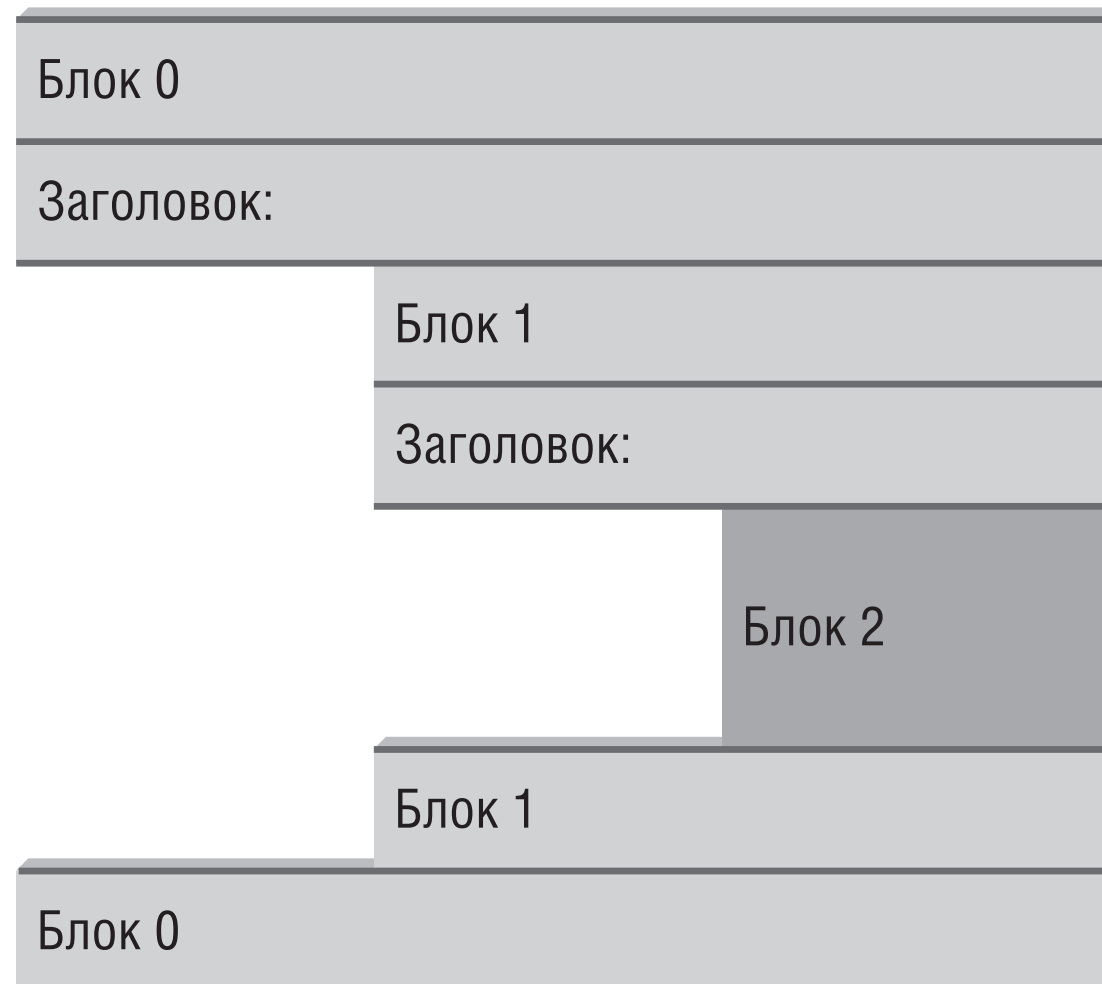
```
>>> if 1:
...     print 'true'
...
true
```

```
>>> if not 1:
...     print 'true'
... else:
...     print 'false'
...
false
```

```
>>> x = 'killer rabbit'
>>> if x == 'roger':
...     print "how's jessica?"
... elif x == 'bugs':
...     print "what's up doc?"
... else:
...     print 'Run away! Run away!'
...
Run away! Run away!
```

Внимание! вложенные блоки

```
x = 1
if x:
    y = 2
    if y:
        print 'block2'
        print 'block1'
print 'block0'
```



Не смешивать символы табуляции и пробелы

<https://youtu.be/tSIfHvgVeQg>

Цикл for

```
for <target> in <object>:    # Связывает элементы объекта с переменной цикла
    <statements>            # Повторяющееся тело цикла: использует переменную цикла
else:
    <statements>            # Если не попали на инструкцию 'break'

for <target> in <object>:    # Присваивает элементы объекта с переменной цикла
    <statements>
    if <test>: break        # Выход из цикла, минуя блок else
    if <test>: continue    # Переход в начало цикла
else:
    <statements>            # Если не была вызвана инструкция 'break'
```

Применение цикла for

```
>>> for x in ["spam", "eggs", "ham"]:  
...     print(x, end=' ')  
...  
spam eggs ham
```

```
>>> sum = 0  
>>> for x in [1, 2, 3, 4]:  
...     sum = sum + x  
...  
>>> sum  
10
```

```
>>> prod = 1  
>>> for item in [1, 2, 3, 4]: prod *= item  
...  
>>> prod  
24
```

Применение цикла for

```
>>> T = [(1, 2), (3, 4), (5, 6)]
>>> for (a, b) in T:           # Операция присваивания кортежа в действии
...     print(a, b)
...
1 2
3 4
5 6

>>> D = {'a': 1, 'b': 2, 'c': 3}
>>> for key in D:
...     print(key, '=>', D[key])   # Используется итератор словаря
...                               # и операция индексирования
a => 1
c => 3
b => 2

>>> list(D.items())
[('a', 1), ('c', 3), ('b', 2)]

>>> for (key, value) in D.items():
...     print(key, '=>', value)   # Обход ключей и значений одновременно
...
a => 1
c => 3
b => 2
```

Применение цикла for

```
>>> T = [(1, 2), (3, 4), (5, 6)]
>>> for (a, b) in T:           # Операция присваивания кортежа в действии
...     print(a, b)
...
1 2
3 4
5 6

>>> D = {'a': 1, 'b': 2, 'c': 3}
>>> for key in D:
...     print(key, '=>', D[key]) # Используется итератор словаря
...                               # и операция индексирования
a => 1
c => 3
b => 2

>>> list(D.items())
[('a', 1), ('c', 3), ('b', 2)]

>>> for (key, value) in D.items():
...     print(key, '=>', value)  # Обход ключей и значений одновременно
...
a => 1
c => 3
b => 2
```

Применение цикла for

```
>>> a, b, c = (1, 2, 3)      # Присваивание кортежа
>>> a, b, c
(1, 2, 3)
>>> for (a, b, c) in [(1, 2, 3), (4, 5, 6)]:    # Используется в цикле for
...     print(a, b, c)
...
1 2 3
4 5 6
```

```
>>> a, *b, c = (1, 2, 3, 4)    # Расширенная инструкция
>>> a, b, c                    # распаковывания последовательностей
(1, [2, 3], 4)
```

```
>>> for (a, *b, c) in [(1, 2, 3, 4), (5, 6, 7, 8)]:
...     print(a, b, c)
...
1 [2, 3] 4
5 [6, 7] 8
```

Применение цикла for

```
>>> items = ["aaa", 111, (4, 5), 2.01] # Множество объектов
>>> tests = [(4, 5), 3.14]             # Ключи, которые требуется отыскать
>>>
>>> for key in tests:                   # Для всех ключей
...     for item in items:             # Для всех элементов
...         if item == key:            # Проверить совпадение
...             print(key, "was found")
...             break
...     else:
...         print(key, "not found!")
...
(4, 5) was found
3.14 not found!
```

«Спрятанный» цикл for

```
vowels = {'a', 'e', 'i', 'o', 'u'}  
message = "Don't forget to pack your towel."
```

```
found = set()  
for v in vowels:  
    if v in message:  
        found.add(v)
```

Генератор множеств
соответствует
тому же шаблону, что
и генератор списков.



`found2 = { v for v in vowels if v in message }`

Обратите внимание: здесь
используются фигурные скобки,
потому что этот генератор
создает множество.

LOGIC



1. Что получится?
2. Как модифицировать код, чтобы получить противоположный результат?

Цикл `while`

```
while <test>:           # Условное выражение test
    <statements1>       # Тело цикла
else:                   # Необязательная часть else
    <statements2>       # Выполняется, если выход из цикла производится не
                        # инструкцией break
```

```
while <test1>:
    <statements1>
    if <test2>: break    # Выйти из цикла, пропустив часть else
    if <test3>: continue # Перейти в начало цикла, к выражению test1
else:
    <statements2>       # Выполняется, если не была использована
                        # инструкция 'break'
```

Применение цикла while

```
>>> while True:
...     print('Type Ctrl-C to stop me!')
```

```
>>> x = 'spam'
>>> while x:           # Пока x не пустая строка
...     print(x, end=' ')
...     x = x[1:]      # Вырезать первый символ из x
...
spam ram am m
```

```
>>> a=0; b=10
>>> while a < b:       # Один из способов организации циклов перечисления
...     print(a, end=' ')
...     a += 1         # Или, a = a + 1
...
0 1 2 3 4 5 6 7 8 9
```

Применение цикла while

```
x = 10
while x:
    x = x-1                # Или, x -= 1
    if x % 2 != 0: continue # Нечетное? - пропустить вывод
    print(x, end=' ')
```

```
x = 10
while x:
    x = x-1
    if x % 2 == 0:         # Четное? - вывести
        print(x, end=' ')
```

```
>>> while 1:
...     name = input('Enter name:')
...     if name == 'stop': break
...     age = input('Enter age: ')
...     print('Hello', name, '=>', int(age) ** 2)
...
Enter name:mel
Enter age: 40
Hello mel => 1600
Enter name:bob
Enter age: 30
Hello bob => 900
Enter name:stop
```

Применение цикла `while`

```
x = y // 2                                # Для значений y > 1
while x > 1:
    if y % x == 0:                         # Остаток
        print(y, 'has factor', x)
        break                             # Перешагнуть блок else
    x -= 1
else:                                     # Нормальное завершение цикла
    print(y, 'is prime')
```

10 лунатиков



Десять лунатиков жили на Луне.
Десять лунатиков ворочались во сне.
И вдруг один лунатик упал с Луны во сне

И девять лунатиков осталось на Луне.
Девять лунатиков жили на Луне.
Девять лунатиков ворочались во сне.
И вдруг один лунатик упал с Луны во сне.

И восемь лунатиков осталось на Луне.
Восемь лунатиков жили на Луне.
Восемь лунатиков ворочались во сне.
И вдруг один лунатик упал с Луны во сне.

...

И один лунатик остался на Луне.
Один лунатик жил на Луне.
Один лунатик ворочался во сне.
И вдруг один лунатик упал с Луны во сне!

И больше лунатиков не стало на Луне